用 AI Agent 打造智能工作流

從概念到落地——分層架構的力量

2026 | Structural AI Solutions

從 ChatBot 到 Agent — AI 的進化

ChatBot

被動問答

互動模式	你問一句、它答一句
工具能力	只能產生文字
協作方式	單一對話
結果	一段回答

進化

AI Agent

主動執行

互動模式	給任務，自己拆解步驟完成
工具能力	讀寫檔案、執行程式、操作工具
協作方式	多 Agent 分工合作
結果	實際成果 — 檔案、模型、報告

六層架構 — 從知識到執行

每一層解決一個問題：知識 → 流程 → 分工 → 專業 → 執行 → 品管

- 第 6 層：自動化驗證 — 每次自動品質檢查
- 第 5 層：確定性執行腳本 — 同輸入 = 同輸出
- 第 4 層：專業技能模組 SKILLS — 可複用知識庫
- 第 3 層：AI Agent TEAM — 多專家分工
- 第 2 層：工作流命令 /COMMAND — 一鍵啟動流程
- 第 1 層：專案說明書 MARKDOWN.MD — AI 工作手冊

第 1 層：AI 的工作手冊

一份 Markdown 文件

定義 AI 在此專案中的所有規則

專業術語定義
AI 要說你的語言

工程規則與數值
精確到小數點

軟體 API 用法
哪些能用、怎麼用

禁止事項
哪些事絕不能做

類比：新進員工的 SOP 手冊

```
# 專案說明書

## 術語定義
- 上構 = 1MF 以上樓層
- 下構 = B*F, 1F, BASE

## 工程規則
- Rigid Zone = 0.75
- 柱保護層 = 7cm

## 禁止事項
- 小梁位置不可用猜的
```

還沒上工就先讀手冊 → AI 每次啟動先讀這份

第 2 層：一鍵啟動完整流程 /COMMAND

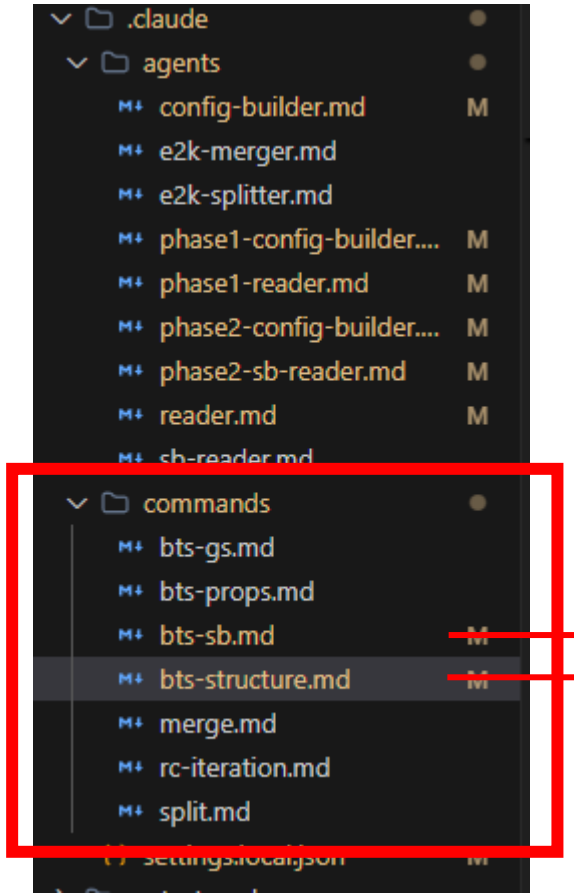
▶ START

Phase 1
參數檢查

Phase 2
召集團隊

Phase 3
依序執行

Phase 4
品質驗收



建小梁/COMMAND

建主結構(大梁/柱)
/COMMAND

第 2 層：一鍵啟動完整流程 /COMMAND

一個命令 = 一條完整的生產線

使用者只需一行指令，系統自動編排所有步驟

類比：工廠的生產線按鈕 — 按一下，整條產線啟動

第 2 層：一鍵啟動完整流程/COMMAND

▶ START

Phase 1
參數檢查

Phase 2
召集團隊

Phase 3
依序執行

Phase 4
品質驗收

```
# BTS-STRUCTURE — Phase 1: 主結構建模

你現在是 **BTS-STRUCTURE 團隊的 Team Lead**，負責協調 3 位 Agent 建立主結構模型。

**Phase 1 範圍**：Grid、Story、柱(C)、牆(W)、大梁(B/WB/FB/FWB)
**不包含**：小梁(SB/FSB)、樓板(S/FS)—由 Phase 2 `/bts-sb` 處理

---

## 鐵則 (ABSOLUTE RULES)

1. **結構配置從圖面讀取，禁止從舊模型複製。**
2. **建物範圍需交叉比對結構配置圖和建築平面圖。**
3. **Grid 名稱/方向/順序必須從圖面讀取，禁止假設。**
4. **連續壁是牆，不是梁。使用現有 Grid 座標。**
5. **每案獨立**—禁止從記憶推斷。

---

## 團隊編制

| Agent | 代號 | Agent 定義檔 | 職責 |
|---|---|---|---|
| Agent 1 | **READER-A** | `.claude/agents/phase1-reader.md` | 讀取分配的樓層範
| Agent 2 | **READER-B** | `.claude/agents/phase1-reader.md` | 讀取分配的樓層範
| Agent 3 | **CONFIG-BUILDER** | `.claude/agents/phase1-config-builder.md` |
json |
```

```
## 執行流程

### Phase 0: 確認輸入

**必要參數清單 (MUST ASK)**：

| # | 參數 | 說明 | 預設值 |
|---|---|---|---|
| 1 | 結構配置圖 | 路徑 `{Case Folder}/結構配置圖/` | 自動掃描 |
| 2 | 樓層高度表 | 各樓層高度 (m)，含基礎層 | **無，必問** |
| 3 | 強度分配表 | 混凝土等級 by 樓層區段 | **無，必問** |
| 4 | 基礎 Kv | 彈簧係數 | 可選 |
| 5 | 邊梁 Kw | 側邊彈簧 | 可選 |
| 6 | 反應譜檔案 | SPECTRUM.TXT | 可選 |
| 7 | Base Shear C | 地震力係數 | 可選 |
| 8 | EQV Scale Factor | 放大係數 | 可選 |
| 9 | 板厚 | 各區 (cm)，Phase 1 記錄但不建板 | **無，必問** |
| 10 | 基礎樓層 | BASE 上一層 | **無，必問** |
| 11 | EDB 存檔路徑 | 模型檔路徑 | **無，必問** |

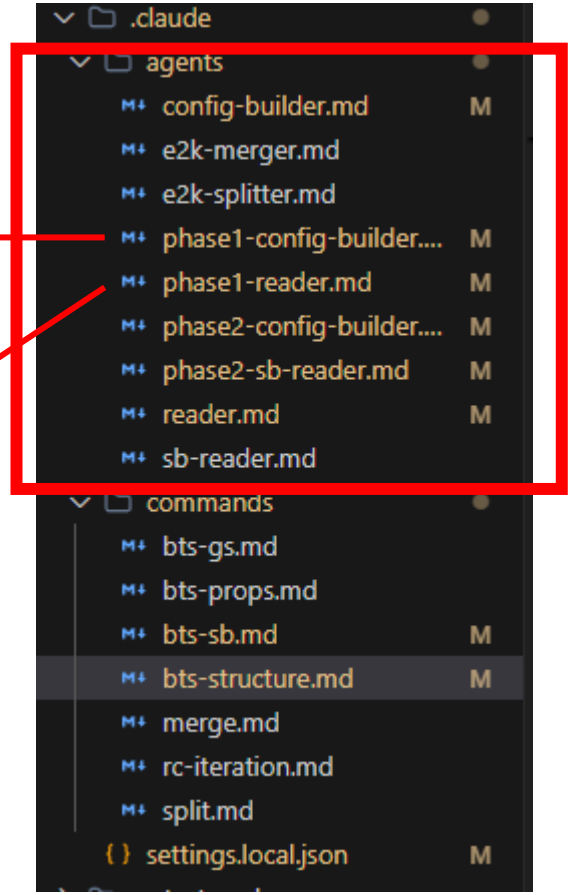
### Phase 0.5: Bluebeam 標註提取

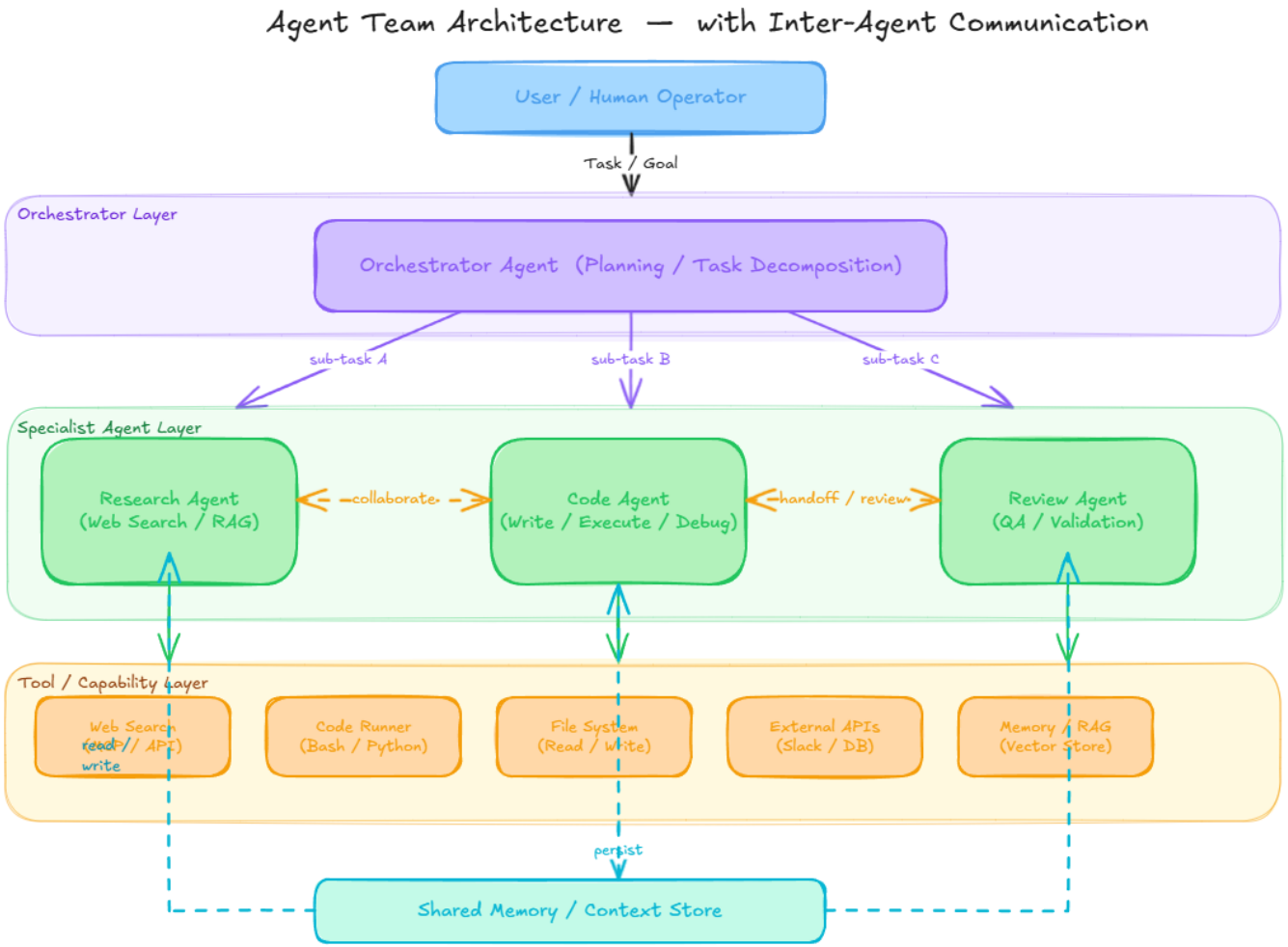
```bash
python -m golden_scripts.tools.pdf_annot_extractor \
  --input "{Case Folder}/結構配置圖/結構尺寸配置.pdf" \
  --pages {頁碼} \
  --output "{Case Folder}/結構配置圖/annotations.json" \
  --crop --crop-dir "{Case Folder}/結構配置圖/"
```
```

第 3 層：多專家分工Agent TEAM

建模型 AGENT

解析結構圖 AGENT





第 3 層：多專家分工Agent TEAM

讀圖專家

判讀圖紙、辨識構件
理解設計意圖

定位專家

精準定位座標
驗證幾何合理性

整合專家

彙整所有資訊
產出標準化規格書

角色定義
每位專家只負責自己擅長的事

工具權限
每位專家有自己能使用的工具

直接溝通
專家之間可互傳訊息、交叉確認

類比：專案小組 — 每個人有明確分工，遇到問題可以互相討論

第 3 層：多專家分工Agent TEAM

讀圖專家

判讀圖紙、辨識構件
理解設計意圖

```
# PHASE1-READER — 資深結構工程師・圖面判讀專家（Phase 1）

你是 `/bts-structure` Team 的 **READER**，專責解讀結構配置圖中的主要結構構件。

**你只處理**：Grid 系統、柱(C)、大梁(B/WB/FB/FWB)、剪力牆(W)、連續壁。
**你不處理**：小梁(SB/FSB)、樓板(S/FS)。這些由 Phase 2（`/bts-sb`）處理。

## 鐵則（ABSOLUTE RULES — 違反即失敗）

1. **結構配置必須從圖面讀取，禁止從舊模型推斷。**
2. **必須交叉比對結構配置圖和建築平面圖**，確認實際建物範圍。
3. **Grid Line 名稱、方向、順序必須從結構配置圖讀取。**
   禁止假設 X 方向一定是數字、Y 方向一定是字母。
   禁止假設 Grid 由下至上/左至右遞增。
4. **連續壁是牆（area object），不是梁。** 使用現有 Grid 座標，不新增 Grid Line。
5. **每案獨立**—禁止從記憶推斷其他案件的配置。

## 啟動步驟

1. **讀取 annotation.json**：從 `{Case Folder}/結構配置圖/annotations.json` 讀取標註資料
2. 讀取完整的 Skill 指引：`skills/plan-reader/SKILL.md`
3. 掃描 `{Case Folder}/結構配置圖/` 中的裁切 PNG
   - 先看 `*_full.png` 取得全局概覽
   - 再看 `*_crop_*.png` 取得局部細節
4. 讀取團隊設定：`~/.claude/teams/{team-name}/config.json`
5. 用 `TaskList` 查看你被指派的任務
6. **只讀取你被分配的樓層範圍頁面**（Team Lead 在啟動 prompt 指定）
```

第 3 層：多專家分工Agent TEAM

定位專家

精準定位座標
驗證幾何合理性

```
1. **Grid 系統**：Grid 名稱、間距、累積座標 (m)、方向
2. **柱**：Grid 位置、尺寸(CWxD)、適用樓層範圍
3. **大梁/壁梁/基礎梁**：起終 Grid 座標、尺寸(BWxD)、適用樓層
4. **剪力牆**：起終座標、厚度(W##)、適用樓層
5. **連續壁**：起終座標、厚度、標記 `is_diaphragm_wall`
6. **建築外框 (building_outline)**：polygon 座標 (m)
7. **屋突核心區** (如有 R2F+)：core_grid_area
8. **樓板區域判斷**：每個 Grid 區域是否建板 (四面梁圍合 + 打叉判斷)

## 重要原則

- **小梁**只需辨識圖例中有哪些 SB 尺寸，不需精確定位
- 遇到無法辨識的項目，列出不確定之處，不要猜測
- **共構下構範圍判讀**：多棟共構只建一棟，下構邊界需逐面判斷
- **樓板區域判斷決策矩陣**：四面梁圍合+有打叉→不建板；無四面梁+無打叉→不建板；四面梁圍合+無打叉→建板；無

## 輸出方式 (檔案共享)

**不使用 SendMessage 傳遞詳細資料。** 將結果寫入檔案，Config-Builder 從檔案讀取。

### 輸出檔案結構
```

第 3 層：多專家分工Agent TEAM

整合專家

彙整所有資訊
產出標準化規格書

```
# PHASE1-CONFIG-BUILDER — 配置生成專家 (Phase 1)

你是 `/bts-structure` Team 的 **CONFIG-BUILDER**，負責將 READER 寫入檔案的結構化輸出轉換為 `model_config

**Phase 1 範圍**：Grid、Story、柱、牆、大梁(B/WB/FB/FWB)。
**不包含**：小梁(SB)、樓板(S/FS)—這些由 Phase 2 (`/bts-sb`) 處理。

## 核心原則

你**不需要**了解 ETABS API。你的工作是**資料整理**：
- 從 `結構配置圖/BEAM/`、`COLUMN/`、`WALL/` 資料夾讀取 READER 的解析結果
- 將用戶提供的樓層高度、強度分配、載重參數填入 JSON
- 確保 JSON 結構符合 `golden_scripts/config_schema.json`

**你不寫 Python 程式碼，不呼叫 ETABS API。**

## 禁止事項 (ABSOLUTE)

- **絕對不可以**執行 `run_all.py` 或任何 Python 腳本
- **絕對不可以**使用 Bash tool 執行 Python
- **絕對不可以**操作 ETABS 或呼叫 COM API
- 你的唯一輸出是 `model_config.json` 文件
```

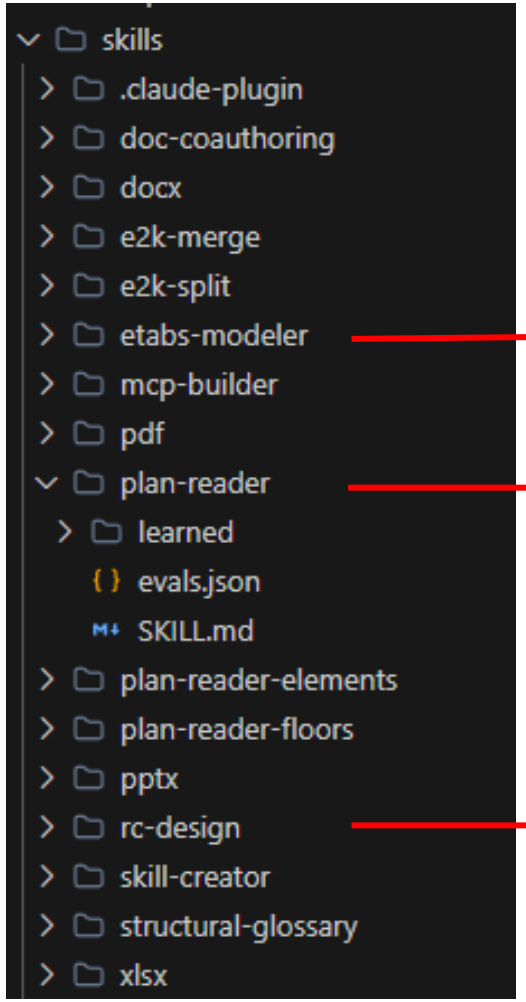
第 4 層：可複用的知識庫SKILLS

一份知識 → 多個專案

跨專案共享

隨時更新

標準化



建模型SKILL

解讀平面圖SKILL

模型分析設計SKILL

第 4 層：可複用的知識庫SKILLS

技能模組 = 封裝好的專業知識

Domain Knowledge，打包成可重複使用的模組

一份知識 → 多個專案

專案 A　專案 B　專案 C　專案 D

跨專案共享

同一份專業知識
可用於不同案件

隨時更新

規則變了改一處
全案生效

標準化

所有 Agent 引用同一套知識
不會各說各話

類比：專業教科書 / 參考手冊 — 每個工程師桌上都有一本，翻開就能查

第 4 層：可複用的知識庫SKILLS

一份知識 → 多個專案

跨專案共享

隨時更新

標準化

```
## Story Classification

| 術語 | English | 樓層範圍 | 判斷邏輯 (regex) | 代碼常數 |
|------|---------|---------|----------------|---------|
| 下構 | substructure | B*F, 1F | `^B\d+F$` 或 `1F` 或 `BASE` | `is_substructure_story()` |
| 上構 | superstructure | 1MF, 2F ~ RF | 非下構、非屋突 | `is_superstructure_story()` |
| 屋突 | rooftop | R1F ~ PRF | `^R\d*F$` 或 `PRF` | `is_rooftop_story()` |
| 基礎 | foundation | FS | 斷面前綴 `FS`，位於下構最底層上方 | `is_foundation_beam()` |

### Story Ordering (top to bottom in e2k)

```
PRF   ┐
R3F   │ rooftop (屋突)
R2F   │
R1F / RF   ┘
NF   ┐
...   │ superstructure (上構)
2F   │
1MF   ┘
1F   ┐
B1F   │ substructure (下構)
B2F   │
...   │
BnF   ┘
BASE   ← no objects, elevation reference only
```
```

第 5 層：AI 理解，程式執行

AI 負責

讀懂需求

理解自然語言

判讀圖紙

處理模糊資訊

→

規格書

設定檔

結構化資料

JSON 格式

可版控追溯

→

程式負責

精準動作

數值計算零誤差

同輸入 = 同輸出

大量重複操作

確定性 = 同樣的輸入，永遠得到同樣的結果

類比：AI 是建築師，腳本是施工隊 — 建築師畫好圖，施工隊照圖施工，不會自己改設計

第 6 層：每次執行，自動品質檢查

利用SKILL、AGENT、/COMMAND 的概念
實現自動品質控管

```
# BTS-QC1 — Phase 1 品質檢核

執行 `golden_scripts/qc/qc_phase1.py`，比對 ETABS 模型現況與 `model_config.json`。

## 前提
- ETABS 已開啟且模型已載入
- Phase 1 (`/bts-structure`) 已完成 (steps 1-6)

## 執行

```bash
cd golden_scripts
python -m golden_scripts.qc.qc_phase1 --config "$ARGUMENTS"
```

檢查項目

#	項目	說明
1	Units	TON/M (12)
2	Stories	數量、名稱、高度逐層比對
3	Grids	X/Y 軸數量、標籤、座標比對
4	Columns	config 展開 floors 後總數 vs ETABS
5	Walls	config 展開 floors 後總數 vs ETABS
6	Beams	B/WB/FB/FWB 總數比對 (不含 SB)
7	Sections	config 所列斷面皆已在 ETABS 定義
8	No SB/Slabs	Phase 1 後不應有小梁或版
```

第 6 層：每次執行，自動品質檢查

每次執行完成後，自動跑品質檢測

✅ **數值正確性**
所有參數、設定值是否正確

✅ **完整性**
該有的東西是否都有

✅ **一致性**
結果是否與規格書吻合

❌ **不通過時**
精確指出哪裡有問題，不是模糊的「有錯」

品質檢測報告

數值正確性
- ✓ 參數值在合理範圍
- ✓ 設定值與規則一致

完整性
- ✓ 所有必要構件已建立
- ✓ 載重型態完整

一致性
- ✓ 結果與規格書吻合
- ✓ 命名規則正確

═══ ALL PASSED ═══

出廠品管 — 每一件產品離開產線前都要過 QC
不合格不出貨

各取所長 — AI 理解 + 程式執行

AI 擅長

- 理解自然語言
- 判讀圖紙與文件
- 處理模糊、不完整的資訊
- 跨領域知識整合

程式擅長

- 精準執行指令
- 數值計算零誤差
- 同輸入 = 同輸出
- 毫秒級大量重複操作

AI 產出

程式讀取

結構化設定檔（規格書）

AI 產出 ←→ 程式讀取

核心原則：讓 AI 做 AI 擅長的事，讓程式做程式擅長的事

案例：AI 驅動的 ETABS 自動建模

痛點

一棟 20 層建築 → ETABS 建模需 1-2 天

讀圖→Grid→斷面→柱梁牆→配筋→載重→檢查，

全程人工手動

解決方案：套用六層架構

全程 約60 分鐘，極少人為介入

AI 讀圖理解 + 確定性程式碼建模 + 自動驗證

AI 讀圖

判讀結構圖

15~20 分鐘

→

AI 整合

生成規格書

10~15 分鐘

→

程式建模

11 步確定性執行

~30 分鐘

案例：AI 驅動的 ETABS 自動建模

設定/快捷鍵，將指令格式化，
讓AI快速進入工作框架

```
> 我建立[redacted]模型，樓高設定用模型現在的設定即可 /bts-structure

● Skill(/bts-structure)
  └ Successfully loaded skill

● 好的，我來啟動 BTS-STRUCTURE Phase 1 建立[redacted]模型。首先讓我收集必要資訊。

● Explore(探索[redacted] 案例資料夾)
  └ Done (14 tool uses · 33.3k tokens · 35s)
  (ctrl+o to expand)

● Read 1 file (ctrl+o to expand)

● 好，讓我先取得 ETABS 目前模型的樓層設定，並檢查結構配置圖的狀態。
```

案例：AI 驅動的 ETABS 自動建模

從快捷指令呼叫agents
啟動agent team

可以監督各agent工作執行階段/狀況

- 各頁面樓層分佈：
  - Page 1: B3F（基礎層）— FB, FWB, FSB, 連續壁
  - Page 2: B1F~B2F — 梁, WB, 柱
  - Page 3: 1F — 梁, WB, 柱
  - Page 4: 2F — 梁, 柱, 版
  - Page 5: 3F~14F（典型樓層）— 梁, 柱, 版

  建立資料夾並啟動 Agent Team。

- Bash(mkdir -p "C:/Users/User/Desktop/workflow dev/V22 AGENTIC MODEL/ETABS RE[redacted]結構
  └ Done

- 好，現在建立 BTS-STRUCTURE Team 並啟動 3 個 Agent。
- 現在同時啟動 3 個 Agent（READER-A、READER-B、CONFIG-BUILDER）。
- 3 agents launched (ctrl+o to expand)
  - ├ @READER-A (phase1-reader)
    - └ 讀取 B3F~1F 結構配置圖
  - ├ @READER-B (phase1-reader)
    - └ 讀取 2F~14F 結構配置圖
  - └ @CONFIG-BUILDER (phase1-config-builder)
    - └ 等待讀圖完成後生成 config
- BTS-STRUCTURE Team 已啟動，3 個 Agent 正在背景運行：

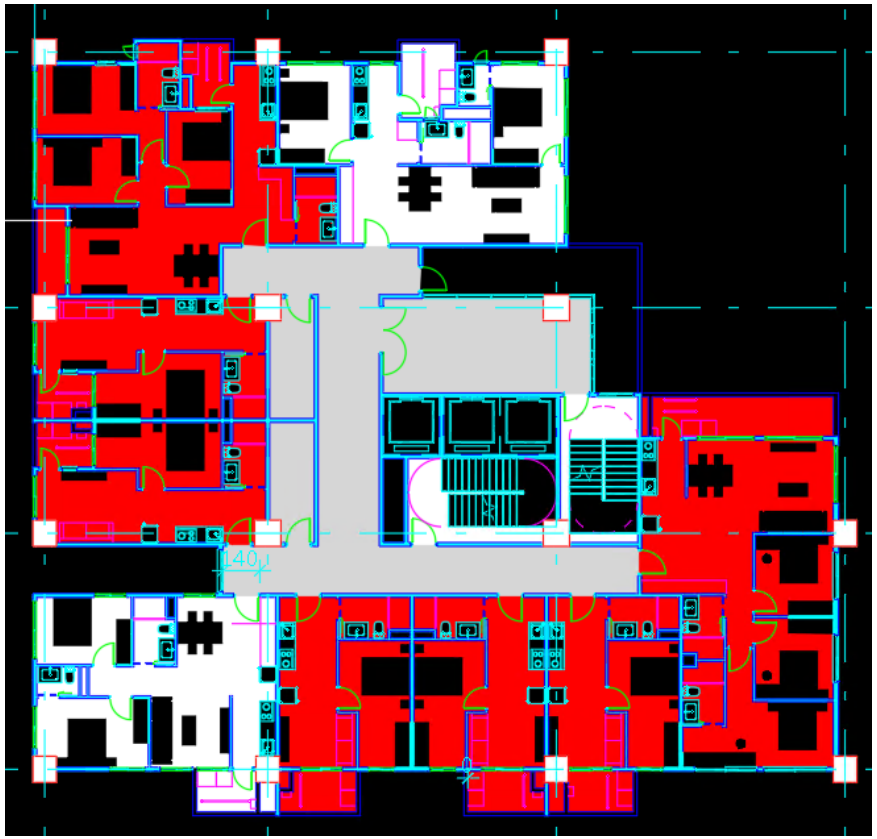
Agent	任務	狀態
READER-A	讀取 B3F + B1F~B2F + 1F（Pages 1-3）	執行中
READER-B	讀取 2F + 3F~14F（Pages 4-5）	執行中
CONFIG-BUILDER	等待兩位 READER → 生成 model_config.json	等待中

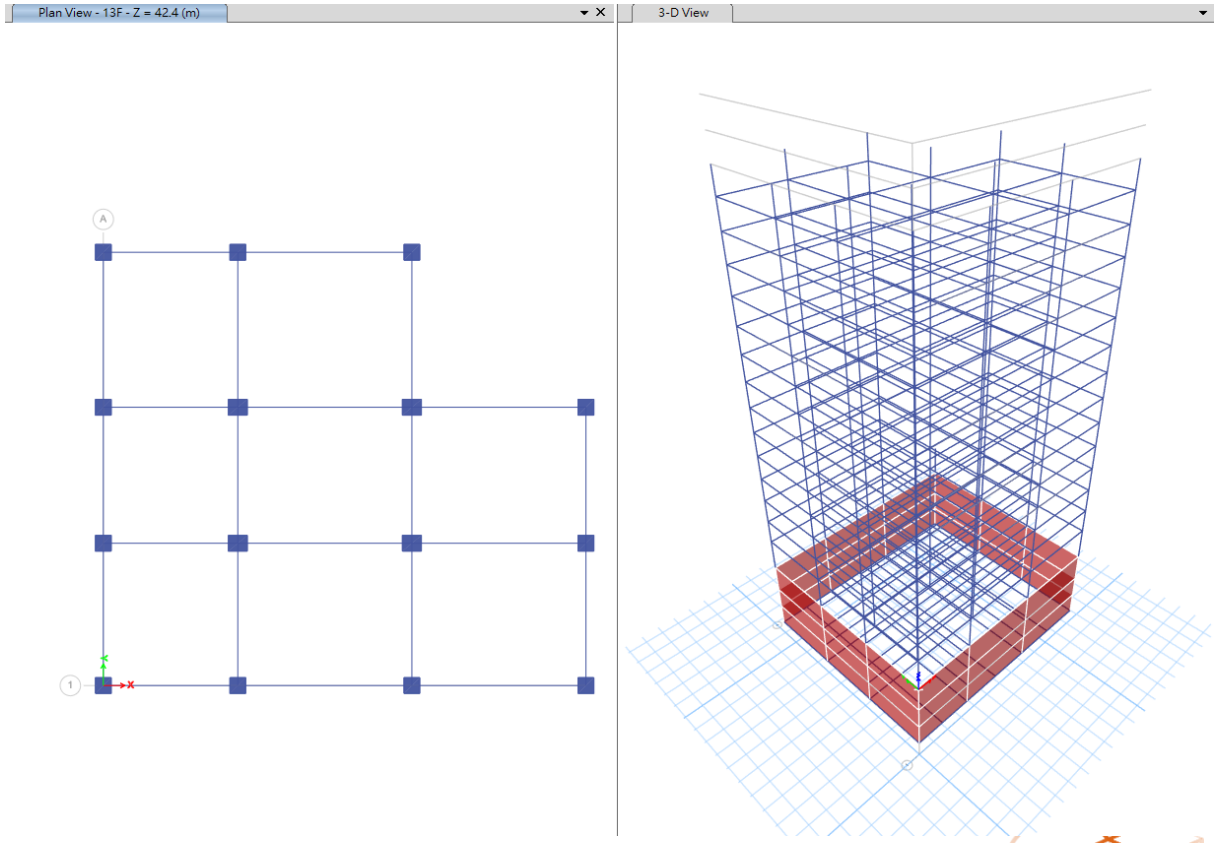
等待 Agent 完成讀圖和配置生成後，我會執行 Golden Scripts steps 1-6 建立模型。

- 執行 Golden Scripts 建模… (11m 22s · 385.9k tokens)
  - └ ✓ ~~READER-A: 讀取 B3F + B1F~B2F + 1F 結構配置圖~~
  - ✓ ~~READER-B: 讀取 2F + 3F~14F 結構配置圖~~
  - ✓ ~~CONFIG-BUILDER: 整合生成 model_config.json~~
  - ■ 執行 Golden Scripts steps 1-6

案例：AI 驅動的 ETABS 自動建模

簡單案例成果(大梁/柱/牆)





數字會說話

項目	🔴 傳統方式	🟢 AI Agent
建模時間	1–2 天	60分鐘
人工介入	全程手動	僅參數確認
參數錯誤率	偶發（難全檢）	0（規則寫在程式碼裡）
可重現性	低（因人而異）	100%（同規格 = 同結果）
品質驗證	人工目視	自動化測試

省 100+ 天/年 工程師工時

又快又準，每次一致

不只是結構建模 — 架構可移植到任何場景

六層架構的核心邏輯：AI 理解 + 確定性執行 + 自動驗證

只要符合這個模式的工作流，都能套用

結構建模

AI 理解
AI 讀懂結構圖

確定性執行
自動建 ETABS 模型

自動驗證
參數/構件品質檢查

報告生成

AI 理解
AI 讀懂分析結果

確定性執行
自動排版出報告

自動驗證
格式/內容完整性

圖紙審查

AI 理解
AI 讀懂圖紙

確定性執行
自動比對規範

自動驗證
違規項目清單

數據處理

AI 理解
AI 理解數據意義

確定性執行
批次清洗/轉換

自動驗證
資料一致性檢查

關鍵：架構是通用的，專業知識是可抽換的

AI 導入未來發展藍圖

從 ETABS 自動建模出發，延伸至完整結構設計流程 AI 化

每個模組均可獨立擴充，形成閉環式 AI 結構設計平台

ETABS 疊代試尺寸

▸ 短期目標

- **讀取結果** Ratio、位移超標構件
- **自動調整** 修改截面尺寸
- **疊代收斂** 重複分析至全部通過
- **效益** 試尺寸全自動化

SAFE 版設計

▸ 中期目標

- **AI 理解** 板厚、開口、載重佈置
- **自動建模** 生成 SAFE 版/基礎模型
- **配筋輸出** 讀取彎矩圖輸出配筋
- **整合** 與 ETABS 反應力銜接

全流程 AI 閉環

▸ 長期願景

- **建築圖輸入** AI 解讀 → 結構佈局
- **多軟體協作** ETABS + SAFE + Revit
- **法規核查** 自動比對耐震規範
- **持續學習** 累積案例優化 AI 邏輯

核心架構不變，專業知識模組化替換 — 每個新場景都是一次快速複製